

1 Analisi dei dati raccolti

I dati raccolti non verranno analizzati solo in valore assoluto, ma verranno **normalizzati rispetto alla propria baseline**. Questo approccio permette di rispondere a domande del tipo: "Quale architettura beneficia maggiormente di un aumento della cache?" o "Quale ISA risulta più efficiente a parità di risorse?".

Il confronto finale avverrà dunque su due livelli:

1. **Intra-ISA:** Confronto tra le tre build della stessa architettura per valutarne la scalabilità.
2. **Inter-ISA:** Confronto tra i rapporti di performance delle tre architetture rispetto alle rispettive baseline, per determinare l'efficienza relativa dei set di istruzioni.

1.1 Analisi delle tre configurazioni per l'architettura ARM

L'analisi delle statistiche relative all'esecuzione del workload su gem5/ARM ha rivelato differenze significative nelle prestazioni e nell'efficienza del sistema, dovute sia alla microarchitettura della CPU, che alla gerarchia di memoria.

<i>Metrica</i>	<i>Baseline (Arm O3)</i>	<i>ArmMinorCPU</i>	<i>Small Caches (O3)</i>
Tempo di simulazione (s)	0,001186	0,005446	0,002132
Istruzioni Simulate	7.807.648	7.807.650	7.807.648
Cicli CPU	2.371.349	10.891.211	4.263.001
IPC (Instructions Per Cycle)	3,292	0,717	1,831
CPI (Cycles Per Instruction)	0,304	1,395	0,546
L1 D-Cache Hits	2.975.995	4.024.561	2.737.002
L1 D-Cache Misses	8943	1878	264.751
L1 D-Cache Miss Rate	0,30%	0,05%	8,82%
L1 I-Cache Hits	572.369	1.113.779	589.353
L1 I-Cache Misses	462	413	468
L1 I-Cache Misses Rate	0,08%	0,04%	0,08%
L2 Cache Hits	60	80	254.708
L2 Cache Misses	2096	2134	2096
L2 Cache Misses Rate	97,22%	96,39%	0,82%
Branch Cond. Incorrect	4637	4715	4636
Branch Mispredicts (Commit)	4548	N/D*	4548

Table 1: Dati di analisi per ARM

1.1.1 Statistiche generali su CPU e istruzioni

La differenza prestazionale più marcata risiede nella scelta della microarchitettura del core. Il modello **ArmO3** (Out-of-Order) risulta circa 4,6 **volte più veloce** rispetto al modello **Minor** (In-Order). Questa disparità è chiaramente riflessa nel valore dell'IPC (Instructions Per Cycle), che crolla da un ottimo 3,29 nella configurazione baseline a un modesto 0,72 nel caso del core Minor.

Il core O3 riesce a superare la barriera delle 3 istruzioni per ciclo sfruttando l'esecuzione fuori ordine e il parallelismo a livello di istruzione (ILP). Al contrario, il core Minor fatica a

completare più di una singola istruzione per ciclo a causa della sua natura *in-order* e di risorse di *issue* limitate; ciò si traduce in un tempo di esecuzione totale significativamente più lungo a parità di carico di lavoro. Nella configurazione con **cache ridotte**, nonostante la CPU rimanga la stessa (O3), si osserva un degrado dell'IPC che scende a 1,83. Questo dimezzamento delle prestazioni è dovuto ai frequenti stalli della pipeline in attesa dei dati recuperati dalla cache L2, la cui latenza è superiore a quella della L1.

1.1.2 Statistiche Cache

Cache hit e Cache Miss L1

L'utilizzo di cache ridotte trasforma la memoria in un collo di bottiglia: il *miss rate* della L1 Data Cache balza dallo 0,3% all'8,8%, con un incremento dei miss di circa 30 volte. Curiosamente, la Minor CPU registra un numero di *hit* totali molto più alto rispetto alla O3. Questo non dipende dal numero di istruzioni (che resta identico), ma dal fatto che il modello *in-order* può generare accessi ripetuti o gestire il *front-end* con *fetch* più frequenti e meno aggregati rispetto alla struttura complessa della O3.

La **I-Cache** (Instruction Cache) rimane invece estremamente stabile e con miss bassissimi in tutte le configurazioni, suggerendo che il codice del workload sia molto compatto e risieda agevolmente anche in spazi ridotti.

Cache hit e Cache Miss L2

L'inefficienza della L1 nelle configurazioni con cache ridotte sposta il carico sulla L2, dove gli *hit* passano da appena 60 a oltre 254.000. Il numero di *L2 Misses* assoluti, tuttavia, resta pressoché identico in tutti i casi (circa 2100). Questo indica che il *working set* totale del programma entra perfettamente nella L2 e che i miss registrati sono esclusivamente **Compulsory Misses** (dati caricati dalla DRAM solo all'inizio).

Si osserva un paradosso statistico nel *miss rate* della L2: esso appare altissimo nella baseline (97,22%) e bassissimo nel caso *small cache* (0,82%). Questo accade perché nella baseline la L1 filtra quasi tutto, mandando in L2 solo le richieste "impossibili" (che falliscono); nel caso delle cache ridotte, la L1 manda in L2 una quantità enorme di richieste "facili" che hanno successo, diluendo il numero di miss e abbassando artificialmente la percentuale, nonostante il sistema stia operando più lentamente a causa della latenza superiore della L2.

1.1.3 Analisi dei Branch

Le statistiche sulla predizione dei rami mostrano valori simili tra le configurazioni O3 (4548 mispredizioni), confermando l'identità del percorso di esecuzione del carico di lavoro. Il modello Minor CPU presenta un numero leggermente superiore di previsioni errate (4715), probabilmente a causa di un hardware di predizione meno sofisticato.

È fondamentale distinguere tra le metriche riportate:

- **Cond. Incorrect:** indica quante volte il predittore ha sbagliato la direzione di un salto condizionale (derivante da istruzioni come *if* o cicli).
- **Branch Mispredicts (Commit):** è un dato globale che include tutti i tipi di salti (ritorni di funzione, salti indiretti, ecc.) che hanno completato l'intera pipeline.

Nella tabella, la voce *Commit* per **ArmMinorCPU** è contrassegnata come **N/D*** per ragioni architettoniche. Nelle CPU O3, il *Commit* (o *Retire*) è l'ultima fase della pipeline, dove i

risultati diventano permanenti e il processore è certo che l'istruzione dovesse essere eseguita. Le CPU O3 usano la **speculazione**: scommettono sulla direzione di un salto e lavorano preventivamente. Se la scommessa è errata, il lavoro viene "buttato via" (*squash*). La statistica di *Commit* conta solo gli errori che arrivano alla fine di questa catena, rappresentando l'impatto reale sulle prestazioni.

Nella CPU Minor, essendo *in-order*, non esiste uno stadio di *Commit* separato: le istruzioni fluiscono linearmente e, se un salto è errato, la pipeline viene resettata immediatamente nello stadio di *Execute*. Pertanto, gem5 utilizza il parametro *condIncorrect* per la Minor, mentre per la O3 il dato di *Commit* risulta più preciso per misurare il rallentamento effettivo. Il "costo" di ogni errore di predizione aumenta drasticamente con cache piccole: se il dato per risolvere il salto è in L2 anziché in L1, la CPU O3 specula "al buio" per più tempo lungo il percorso errato, caricando istruzioni inutili e sprecando cicli.

1.1.4 Conclusioni

L'analisi comparativa tra le diverse configurazioni architetturali e gerarchie di memoria permette di trarre conclusioni fondamentali sul bilanciamento di un sistema di calcolo:

- **Dominanza dell'architettura Out-of-Order:** per il carico di lavoro analizzato, il core O3 si è dimostrato superiore in termini di throughput. Il parallelismo a livello di istruzione (ILP) estratto dal core compensa ampiamente la complessità hardware, permettendo di completare il task in una frazione del tempo richiesto dal core Minor;
- **Criticità della Cache L1:** la dimensione della cache di primo livello è il vero ago della bilancia prestazionale. Il workload presenta un *working set* che risiede agevolmente nella L1 standard, ma l'impiego di una cache ridotta causa un effetto a catena: l'incremento del miss rate sposta il carico sulla L2, introducendo latenze che nemmeno un core avanzato come l'O3 riesce a mascherare, portando a un rallentamento medio del 45% (fino a rendere l'O3 meno efficiente di un core In-Order in ambito RISC-V);
- **Limiti del core In-Order:** il modello Minor è strutturalmente limitato dal proprio front-end. La sua natura *single-issue* non permette di sfruttare appieno le unità di esecuzione anche quando i dati sono disponibili in cache, rendendolo una scelta valida solo in contesti di estremo risparmio energetico o dove le risorse di memoria sono talmente scarse da annullare i benefici della speculazione;
- **Saturazione della Gerarchia:** i circa 2100 miss costanti in L2 (per ARM) e 3010 (per RISC-V) rappresentano i **Compulsory Misses** del programma. Questo dato indica che il limite fisico minimo di accessi alla memoria principale (DRAM) è intrinseco al codice stesso e rimane invariato indipendentemente dalla potenza del processore o dalla dimensione delle cache intermedie.

In conclusione, l'esperimento evidenzia che una CPU potente è efficace solo se supportata da un sistema di memoria equilibrato. La scelta dell'architettura deve quindi basarsi non solo sulla velocità teorica del core, ma sulla capacità della gerarchia di memoria di alimentare costantemente la pipeline, evitando che la speculazione diventi un costo anziché un vantaggio.

1.2 Analisi delle tre configurazioni per l'architettura RISC-V

<i>Metrica</i>	<i>Baseline (RISCV O3)</i>	<i>Minor_RISCV</i>	<i>Small Caches (O3)</i>
Tempo di simulazione (s)	0,003567	0,005997	0,006364
Istruzioni Simulate	10.043.070	10.043.070	10.043.070
Cicli CPU	7.134.755	11.994.951	12.728.034
IPC (Instructions Per Cycle)	1,408	0,837	0,789
CPI (Cycles Per Instruction)	0,710	1,194	1,267
L1 D-Cache Hits	3.551.546	4.050.839	3.315.652
L1 D-Cache Misses	16.645	3681	252.600
L1 D-Cache Miss Rate	0,47%	0,09%	7,08%
L1 I-Cache Hits	600.802	1.150.882	600.828
L1 I-Cache Misses	619	537	629
L1 I-Cache Misses Rate	0,10%	0,05%	0,10%
L2 Cache Hits	40	40	235.932
L2 Cache Misses	3010	3056	3010
L2 Cache Misses Rate	98,69%	98,71%	1,26%
Branch Cond. Incorrect	4816	4898	4817
Branch Mispredicts (Commit)	4725	N/D*	4729

Table 2: Dati di analisi per RISC-V

1.2.1 Statistiche generali su CPU e istruzioni

Nell'architettura RISC-V, la CPU **O3** (Out-of-Order) si dimostra circa **1,7 volte più veloce** della **Minor CPU** in termini di tempo di simulazione nella configurazione baseline. L'IPC della baseline risulta pari a 1,408, un valore che evidenzia la capacità del core di processare più di un'istruzione per ciclo grazie all'esecuzione fuori ordine e al parallelismo a livello di istruzione (ILP). Al contrario, il core Minor, vincolato dall'esecuzione sequenziale (*in-order*), subisce maggiormente l'impatto di stalli strutturali e dipendenze tra i dati, portando il tempo di esecuzione a quasi 6 ms contro i 3,5ms della baseline.

Tuttavia, l'efficienza del core O3 è strettamente legata alla gerarchia di memoria. Nella configurazione con **cache ridotte**, la CPU O3 subisce un degrado tale da diventare la più lenta delle tre configurazioni (6,36 ms), risultando persino meno efficiente del modello Minor. Questo accade perché l'architettura fuori ordine è estremamente sensibile alla latenza: quando la L1 D-Cache diventa un collo di bottiglia (con un *miss rate* che balza dallo 0,47% al 7,08%), le strutture interne della CPU si saturano in attesa dei dati. In questo scenario, l'IPC crolla a 0,789 e la complessità del core O3 diventa un onere, poiché il processore occupa più tempo a gestire stalli e speculazioni fallite che a eseguire codice utile.

1.2.2 Statistiche Cache

Cache hit e Cache Miss L1

L'analisi dei flussi rivela una **I-Cache** (Instruction Cache) estremamente stabile in tutte le configurazioni (miss rate 0,10%), indicando che il codice eseguibile è compatto e risiede agevolmente in memoria.

La criticità è invece concentrata sulla **L1 D-Cache**: nella baseline essa filtra la quasi totalità delle richieste, ma nella versione *small cache* subisce un'espulsione massiccia di dati, portando il numero di miss da circa 16k a oltre 252k.

Cache hit e Cache Miss L2

Nella configurazione baseline, la L2 riceve solo circa 3000 richieste, che falliscono quasi integralmente (*miss rate* 98,7%). Si tratta di **Compulsory Misses**, ovvero il caricamento iniziale di dati mai visti prima. Nella versione con cache ridotte, la L2 viene invece investita da oltre 235.000 accessi (hits) precedentemente gestiti dalla L1. Sebbene il *miss rate* della L2 scenda paradossalmente all'1,26%, ciò riflette solo il fatto che la L2 sta supplendo alle carenze della L1, introducendo però una latenza d'accesso superiore che penalizza l'intero sistema.

1.2.3 Analisi dei Branch

Il numero di predizioni errate è molto simile tra le due versioni O3 (4729 mispredicts al commit), confermando che il percorso logico del software non cambia. Nella configurazione con cache ridotte, pur restando costante il numero di errori, la CPU paga un prezzo molto più alto in termini di cicli di stallo, poiché i miss in L1 ritardano il recupero dei dati necessari a risolvere la condizione del salto.

Confronto O3 vs Minor e impatto della latenza

Analizzando la distinzione tra *condIncorrect* (errori sui salti condizionali) e *commit.branchMispredicts* (dato globale al ritiro), si osserva che:

- **Configurazione Minor**: come previsto, la voce *Commit* riporta N/D* per l'assenza di uno stadio di ritiro separato. Il valore di *condIncorrect* (4898) è leggermente superiore ai modelli O3, suggerendo che il predittore integrato nel core Minor sia meno raffinato.
- **Latenza di risoluzione**: nella versione con cache ridotte, l'alto tasso di miss in L1 D-Cache rallenta drasticamente il calcolo delle condizioni nello stadio di *Execute*. La CPU rimane in uno stato di "incertezza speculativa" per un tempo prolungato, causando un incremento delle istruzioni *squashed* (caricate inutilmente lungo percorsi errati) che contribuisce al crollo dell'IPC.

1.2.4 Conclusioni

L'analisi comparativa su architettura RISC-V mette in luce dinamiche differenti rispetto a quanto osservato per ARM, evidenziando un equilibrio ancora più precario tra potenza di calcolo e sottosistema di memoria:

- **Superiorità condizionata dell'architettura Out-of-Order**: nella configurazione baseline, il core **O3** si dimostra circa 1,7 volte più veloce del modello Minor. Tuttavia, l'IPC di 1,408 suggerisce che il parallelismo estratto (ILP) sia inferiore rispetto ad altre architetture, rendendo il vantaggio della speculazione meno netto e più dipendente dall'efficienza delle cache.
- **Il "sorpasso" della Minor CPU**: a differenza di ARM, in RISC-V la criticità della cache L1 è tale da ribaltare le gerarchie di potenza. Nella configurazione **Small Caches**, il core O3 diventa il meno efficiente del gruppo (6,36ms), venendo superato in termini di tempo di esecuzione persino dal core **Minor** (5,99ms). Questo dimostra che, se la memoria non è in grado di alimentare la pipeline, la complessità dell'esecuzione fuori ordine diventa un

ostacolo anziché un beneficio.

- **Prevedibilità del core In-Order:** il modello Minor, pur essendo limitato dalla sua natura *single-issue* e dalla mancanza di uno stadio di commit complesso, si dimostra più resiliente alle variazioni del sistema di memoria. In scenari con risorse limitate, la sua esecuzione lineare evita i pesanti costi di "svuotamento" della pipeline (squash) che affliggono il core O3 quando la latenza di risoluzione dei branch aumenta.
- **Saturazione della Gerarchia:** i circa 3010 miss costanti registrati in L2 rappresentano i **Compulsory Misses** del carico di lavoro RISC-V. Questo valore indica la soglia minima di accessi alla DRAM necessaria per caricare il set di dati iniziale; il fatto che tale numero resti invariato conferma che il collo di bottiglia prestazionale risiede esclusivamente nella latenza di accesso ai dati (L1 vs L2) e non nel volume totale dei dati stessi.

In conclusione, l'esperimento su RISC-V sottolinea un principio fondamentale della progettazione hardware: **un core complesso richiede una gerarchia di memoria proporzionata**. Senza una cache L1 adeguata, l'architettura O3 "affama" se stessa, rendendo paradossalmente preferibile una microarchitettura più semplice e lineare come quella del core Minor.

1.3 Analisi delle tre configurazioni per l'architettura MarssX86

L'analisi condotta sull'architettura x86 tramite il simulatore Marss ha l'obiettivo di isolare il comportamento del workload (User) rispetto all'overhead introdotto dal sistema (Total) al variare delle configurazioni microarchitetture, poiché MarssX86 è un emulatore Full System (FS).

<i>Metrica</i>	<i>Baseline (OoO)</i>	<i>Var 1: Atom</i>	<i>Var 2: Small Caches</i>
Tempo Host (s)	9	6	9
Tempo simulato (ms)	4,6409	8,4707	4,8709
Cicli CPU	12.474.800	22.769.162	13.092.853
Istruzioni	7.760.158	7.448.858	7.981.476
IPC	0,6221	0,3271	0,6096
CPI	1,6075	3,0567	1,6404
L1i Hit	1.191	1.641	4.490
L1i Miss	519	482	3.236
L1i Miss Rate	30,35%	22,70%	41,88%
L1d Hit	3.273.198	4.253.740	3.292.506
L1d Miss	1.869	1.847	43.010
L1d Miss Rate	0,06%	0,04%	1,29%
L2 Hit	56	71	41.712
L2 Miss	2.332	2.258	4.534
L2 Miss Rate	97,65%	96,95%	9,80%
Branch Mispred.	4.561	5.020	6.567
Branch Cond. Inc.	4.539	5.020	6.408

Table 3: Confronto metriche generali di simulazione (Total)

<i>Metrica</i>	<i>Baseline (OoO)</i>	<i>Var 1: Atom</i>	<i>Var 2: Small Caches</i>
Tempo simulato (ms)	4,6123	8,4273	4,6774
Cicli CPU	12.397.779	22.652.712	12.572.917
Istruzioni	7.733.613	7.430.872	7.733.133
IPC	0,6238	0,3280	0,6151
CPI	1,6031	3,0485	1,6259
L1i Hit	1.191	61	4.490
L1i Miss	519	20	3.236
L1i Miss Rate	30,35%	24,69%	41,88%
L1d Hit	3.265.122	4.247.880	3.226.082
L1d Miss	1.139	1.201	40.130
L1d Miss Rate	0,03%	0,03%	1,23%
L2 Hit	21	68	40.368
L2 Miss	1.637	1.153	2.998
L2 Miss Rate	98,73%	94,43%	6,91%
Branch Mispredict (commit)	4.199	4.182	4.176
Branch Cond. Inc.	4.194	4.182	4.172

Table 4: Tabella USER (Solo Workload)

Note: Questa tabella isola l'efficienza pura dell'algoritmo di moltiplicazione matrici.

1.3.1 Statistiche generali su CPU e istruzioni

Il primo confronto riguarda l'efficienza di esecuzione delle tre varianti: Baseline (Out-of-Order), Atom (In-Order) e Small Caches (Out-of-Order con cache ridotta).

Osservando i dati User, che rappresentano il puro carico computazionale della moltiplicazione di matrici, notiamo che il numero di istruzioni rimane costante (~ 7.73 milioni), confermando la correttezza deterministica della simulazione. Tuttavia, i Cicli CPU variano drasticamente:

- La **configurazione Atom** richiede il 82% di cicli in più (22.6M) rispetto alla Baseline (12.4M) per eseguire lo stesso codice. Questo crollo dell'IPC (da 0,62 a 0,33) è intrinseco alla natura In-Order della pipeline, che non può mascherare le latenze di esecuzione tramite l'esecuzione fuori ordine.
- La **configurazione Small Caches**, nonostante la drastica riduzione della memoria, mostra una resilienza notevole: l'IPC User cala solo marginalmente (0,615 vs 0,624). Questo dimostra che il motore Out-of-Order riesce a nascondere efficacemente le latenze aggiuntive introdotte dal sottosistema di memoria.

Un dato interessante riguarda il Tempo di Simulazione (Host): la configurazione Atom completa la simulazione in meno tempo (6s) rispetto alle versioni OoO (9s). Ciò è dovuto alla minore complessità del modello matematico della CPU In-Order, che non deve simulare strutture complesse come il Reorder Buffer o la rinomina dei registri.

1.3.2 Statistiche Cache

L'analisi della gerarchia di memoria evidenzia come la dimensione delle cache influenzi il traffico dati tra la CPU e la memoria principale.

Cache hit e Cache Miss L1

La Cache L1 Dati (L1d) si rivela il componente critico. Nella Baseline e nella configurazione Atom (128kB), i miss User sono trascurabili (~ 1.100), indicando che l'intero working set delle matrici risiede comodamente nella cache. Nella configurazione Small Caches (32kB), invece, assistiamo a un fenomeno di *capacity miss*: i miss User salgono a 40.130.

La cache è troppo piccola per contenere i blocchi di matrice riutilizzati dall'algoritmo, costringendo la CPU a continue espulsioni e ricaricamenti.

È interessante notare come il core Atom generi circa 1 milione di accessi L1 in più rispetto alla Baseline: mancando di meccanismi di *Store-to-Load Forwarding*, la CPU In-Order è costretta a formalizzare ogni passaggio di dati attraverso la cache.

Cache hit e Cache Miss L2

La Cache L2 agisce come "rete di salvataggio".

- Nelle **configurazioni con L1 grande (Baseline/Atom)**, la L2 è scarsamente utilizzata (pochi accessi, quasi tutti cold miss), risultando in un miss rate elevato ma irrilevante in termini assoluti.
- Nella **configurazione Small Caches**, la L2 diventa fondamentale. Essa intercetta quasi la totalità dei dati espulsi dalla L1: su 43.000 accessi totali, ben 41.712 sono Hit.

Questo spiega perché le prestazioni della Small Caches non crollano: sebbene la L1 fallisca spesso, i dati vengono recuperati dalla L2 (molto più veloce della RAM), mitigando la penalità temporale.

1.3.3 Analisi dei Branch

L'analisi della predizione dei salti rivela un'interessante correlazione indiretta con la memoria.

Confrontando i dati Total (Sistema completo), si nota un aumento dei *Branch Mispredictions* nella configurazione Small Caches (6.567) rispetto alla Baseline (4.561). Tuttavia, isolando i dati User, il numero di errori rimane pressoché identico (~ 4.180) per tutte le configurazioni.

Questa discrepanza porta a una conclusione importante: Il peggioramento apparente nella predizione dei salti non è dovuto a una difficoltà del processore nel prevedere il flusso del programma di moltiplicazione (che rimane regolare), ma è un effetto collaterale dello stress sulla memoria. L'alto numero di miss in L1 nella versione Small Caches costringe il sistema operativo a intervenire più spesso (gestione page fault, buffer, interrupt). Il codice del Kernel, essendo più irregolare, induce il predittore in errore più frequentemente. Pertanto, il calo di efficienza del front-end è una conseguenza diretta del collo di bottiglia nel back-end (memoria).

1.3.4 Conclusioni

L'analisi condotta sull'architettura MarssX86 permette di trarre tre conclusioni fondamentali riguardo l'interazione tra microarchitettura, memoria e sistema operativo.

In primo luogo, si conferma la **superiorità dell'esecuzione Out-of-Order** per carichi di lavoro *memory-intensive* come la moltiplicazione di matrici. La configurazione Baseline (OoO) ha completato il task con la metà dei cicli rispetto alla configurazione Atom (In-Order), dimostrando come la complessità hardware (rinomina dei registri, esecuzione speculativa)

sia necessaria per mascherare le latenze di accesso ai dati, mantenendo un CPI vicino a 1,6 contro il 3,0 dell'Atom.

In secondo luogo, l'esperimento ha evidenziato la **resilienza della pipeline OoO** di fronte a risorse scarse. Nonostante la riduzione della Cache L1 abbia causato un aumento dei miss di oltre il 2000% (da ~ 1.800 a ~ 43.000), l'IPC del workload utente è rimasto quasi invariato (0,615 vs 0,624).

Questo risultato controintuitivo dimostra che un moderno processore OoO, supportato da una Cache L2 capiente ed efficiente, è in grado di tollerare fallimenti massicci nel primo livello di memoria senza stallare l'esecuzione del programma principale.

Infine, la disaggregazione dei dati tra User e Kernel ha svelato il vero costo del sottodimensionamento della cache: l'**overhead di sistema**. Il degrado delle prestazioni globali nella configurazione *Small Caches* non è imputabile al rallentamento dell'algoritmo di calcolo, bensì all'esplosione dell'attività del Sistema Operativo (+830% di istruzioni kernel), costretto a intervenire massicciamente per gestire la congestione del sottosistema di memoria.

Ne consegue che, in scenari di progettazione embedded o low-power, il dimensionamento della cache L1 non deve essere valutato solo in funzione del throughput dell'applicazione, ma anche per garantire la stabilità e l'efficienza dei servizi di sistema.

1.4 Analisi delle baseline tra le architetture

<i>Metrica</i>	<i>Baseline (Arm O3)</i>	<i>Baseline (RISC-V)</i>	<i>Baseline user (MarssX86)</i>
Tempo di simulazione	0,001186s	0,003567s	4,6123ms
Istruzioni Simulate	7.807.648	10.043.070	7.449.759
Cicli CPU	2.371.349	7.134.755	12.474.800
IPC (Instructions Per Cycle)	3,292	1,408	0,597
CPI (Cycles Per Instruction)	0,304	0,710	1,675
L1 D-Cache Hits	2.975.995	3.551.546	3.265.122
L1 D-Cache Misses	8.943	16.645	1.139
L1 D-Cache Miss Rate	0,30%	0,47%	0,03%
L1 I-Cache Hits	572.369	600.802	1.191
L1 I-Cache Misses	462	619	519
L1 I-Cache Misses Rate	0,08%	0,10%	30,35%
L2 Cache Hits	60	40	21
L2 Cache Misses	2096	3010	1.637
L2 Cache Misses Rate	97,22%	98,69%	97,6%
Branch Cond. Incorrect	4637	4816	4.199
Branch Mispredicts (Commit)	4548	4725	4.561

Table 5: Tabella delle tre baseline a confronto

Note: Per MarssX86 sono stati isolati i dati relativi al solo workload utente.

1.4.1 Statistiche generali su CPU e istruzioni

L'analisi delle prestazioni generali illustrata in tabella 5, rivela un netto vantaggio per l'architettura *ARM O3*, che svetta con un IPC di 3,292. Questo indica una pipeline estremamente efficiente nel gestire il parallelismo a livello di istruzione (ILP) per il workload di moltiplicazione di matrici, completando il compito in soli 1,18 ms simulati.

Il *RISC-V* si posiziona in una fascia intermedia con un IPC di 1,408. Sebbene esegua un numero maggiore di istruzioni totali (10,04M) rispetto ad ARM, la sua efficienza per ciclo è inferiore, portando a un tempo di simulazione tre volte superiore (3,56 ms).

La baseline di *MarssX86* mostra invece l'IPC più basso (0,597). È fondamentale sottolineare che, avendo isolato la componente *user* (il workload applicativo) dalla componente *kernel* (il sistema operativo), l'IPC registrato riflette la complessità della decodifica x86 e la presenza di stalli nella pipeline che persistono nonostante l'ottimizzazione del codice utente.

1.4.2 Statistiche Cache

L'efficienza del sottosistema di memoria gioca un ruolo fondamentale nel determinare i risultati prestazionali sopra descritti.

Cache hit e Cache Miss L1

Isolando le statistiche del workload (*User mode*) in *MarssX86*, emerge un dato estremamente positivo per la L1 D-Cache: il miss rate scende allo 0,03%, il valore più basso tra le tre architetture. Questo dimostra che l'algoritmo di moltiplicazione di matrici è perfettamente ottimizzato per la gerarchia di memoria x86.

Tuttavia, persiste un miss rate anomalo nella L1 I-Cache (30,35%). Analizzando il numero assoluto di accessi (1.710), si deduce che tale valore è un artefatto statistico: trattandosi di un workload breve, i *cold miss* (miss obbligatori al primo caricamento delle istruzioni) pesano sproporzionatamente sulla statistica finale, a differenza di ARM e RISC-V dove il volume di istruzioni simulate permette di ammortizzare questi costi iniziali.

Cache hit e Cache Miss L2

Per tutte e tre le architetture, il miss rate in L2 è prossimo al 98%. Questo fenomeno è tipico di workload i cui dati risiedono quasi interamente in L1 (128 kB). Poiché la L1 filtra con successo quasi tutti gli accessi del workload utente (come confermato dal miss rate dello 0,03% su x86), alla L2 giungono pochissime richieste, rendendo statisticamente dominante ogni singolo miss dovuto a dati non ancora mappati.

1.4.3 Analisi dei Branch

Nonostante le differenze di throughput, il comportamento del *Branch Predictor* è coerente tra le architetture, con errori di predizione che oscillano tra 4.500 e 4.800.

Come evidenziato dai dati, per i modelli Out-of-Order (O3) il parametro di riferimento è il *Branch Mispredicts (Commit)*. In queste CPU, il processore esegue istruzioni in modo speculativo: se la "scommessa" sul salto è errata, l'errore viene rilevato e contabilizzato solo nella fase finale di **Commit**. I circa 4.500 errori rappresentano i salti che hanno effettivamente causato il *flush* (svuotamento) della pipeline, impattando direttamente sul CPI finale.

In sintesi, l'isolamento della componente *user* in MarssX86 conferma che il collo di bottiglia non risiede nella gestione dei dati (D-Cache estremamente efficiente), ma nella penalità di decodifica x86 e nei miss iniziali di caricamento delle istruzioni, che in un workload così breve condizionano pesantemente l'IPC finale.

1.4.4 Conclusioni

In conclusione, il confronto tra le tre baseline evidenzia come l'architettura **ARM O3** sia la più performante per il workload di calcolo matriciale, grazie a un bilanciamento ottimale tra velocità di clock simulata e gestione della pipeline. Il **RISC-V** si conferma un'architettura solida con un'efficienza predicibile, mentre **MarssX86** rivela che, nonostante un'eccellente gestione dei dati (D-Cache), soffre di un overhead maggiore nella gestione del flusso di istruzioni. L'analisi "User-only" è stata fondamentale per distinguere le reali prestazioni dell'algoritmo dal rumore di fondo generato dal sistema operativo simulato, permettendo un confronto più equo tra le diverse tecnologie.